DS314: GENERATIVE AI & LLM SYSTEMS · ACADEMIC LAB MODULE

DS314 - Lab 2: Building an End-to-End Streamlit RAG Chatbot

Format:	Jupyter Notebook (.ipynb)	**Source:**	DS314_Lab2_Streamlit_RAG.ipynb	**Render Engine:**	KaTeX + MathML (Zero-Delim Math)
Academic Level:	3rd Year BS Data Science				

DS314 -- DS Elective 1 | Generative AI

Laboratory Task 2: Building an End-to-End RAG Chatbot

Topic: Streamlit Core Framework & Developer Guide

Student Information (Pair Work / Buddy System)

- **Partner 1 Name:** Ashlanie L. Pandi
- **Partner 2 Name:** Rogelio Q. Mandamian III
- **Source Document:** `data/streamlit_developer_guide.pdf`

Lab Overview

In this lab, you will build a **Retrieval-Augmented Generation (RAG)** chatbot based on what you learned in **DS314 - Topic 2**.

Normally, large language models might guess or make things up when asked about specific technical documents. With RAG, we first find the relevant parts of our document (`streamlit_developer_guide.pdf`) and pass them to Google's **Gemini** model so it can answer accurately using real, grounded facts.

Tools Used

- **LangChain**: Connects all the parts of our RAG pipeline together.
- **Google Gemini API**:
  - `gemini-embedding-2` : Turns text chunks into embeddings (lists of numbers representing meaning).
  - `gemini-3.5-flash` : Generates the final grounded answers.
- **ChromaDB**: An in-memory vector database that stores and searches text chunks.

Topic Selection: Why We Chose Streamlit

The reason why we chose Streamlit was because I was already planning on learning this, but I always end up putting it off, so this lab was the perfect excuse to finally dive in.

Beyond that, Streamlit is the standard tool used in data science and generative AI to turn Python code into clean, interactive web applications without needing to know HTML, CSS, or JavaScript. It also has very clear and strict technical rules, such as rerunning the entire script from top to bottom on every user click, requiring `st.session_state` to save variables, and having separate caching decorators for data versus resources. These specific mechanics make it an ideal test for our RAG pipeline to see if the chatbot can pull exact technical facts instead of making vague guesses. Plus, having a grounded documentation chatbot will be genuinely useful for us as we build web apps and interfaces in future coursework.

Key Reasons for Choosing Streamlit:

1. **Practical Learning Goal:** We wanted hands-on experience building web apps for AI prototypes and data science projects.
2. **Strict & Nuanced Rules:** Features like top-to-bottom reruns, `st.session_state`, and `@st.cache_data` vs `@st.cache_resource` give our RAG pipeline concrete technical facts to test.
3. **Testing Hallucination Boundaries:** Because many LLMs mix up Streamlit commands with other frameworks, testing questions like `st.deploy()` lets us see if the chatbot stays honest.
4. **Natural Pair Division:** It splits cleanly between Partner 1 (UI elements, widgets, layout columns) and Partner 2 (session state, caching, configuration, trap questions).

Part 1: Setting Up the RAG Pipeline

Step 1.1: Project Setup & Repository Hygiene (`.gitignore`)

As students learning good software engineering practices, our first priority is ensuring sensitive credentials and temporary environment files never get accidentally committed to GitHub.

We set up our `.gitignore` file to automatically ignore the following:

Item in `.gitignore`	Files / Folders	Why We Keep It Out of Git
Environment Credentials	`.env`	Holds our private `GEMINI_API_KEY`. Treat it like a password—never share it publicly.
Python Bytecode & Caches	`__pycache__/`, `*.py[cod]`, `*$py.class`	Compiled Python files created automatically at runtime that would clutter our repo.
ChromaDB Vector Store	`chroma_db/`	The local folder where vector index files are saved; we exclude it to prevent repository bloat.
Jupyter Checkpoints	`.ipynb_checkpoints/`	Auto-saved temporary checkpoint files created by Jupyter while editing.
Virtual Environments	`venv/`, `.venv/`, `env/`	Local Python environments. Other developers install dependencies cleanly from `requirements.txt`.

Step 1.2: Verifying Our Google Gemini API Key

Before making any AI calls, we check that `python-dotenv` can read our `GEMINI_API_KEY` from the local `.env` file.

In [1]:

```python
# Step 1.2: Verify our API key is accessible in our environment
from dotenv import load_dotenv
import os
import sys
import warnings
import logging

# Ensure UTF-8 stdout encoding on Windows
if hasattr(sys.stdout, 'reconfigure'):
    try:
        sys.stdout.reconfigure(encoding='utf-8')
    except Exception:
        pass

# 1. Suppress Python UserWarnings (e.g. fixed sampling parameter notices)
warnings.filterwarnings("ignore")

# 2. Suppress Google GenAI internal logger warnings (e.g. AFC recommendations)
logging.getLogger("google_genai").setLevel(logging.ERROR)
logging.getLogger("google_genai.models").setLevel(logging.ERROR)

# Locate .env regardless of whether notebook is run from root or notebooks/
for env_candidate in [".env", "streamlit-rag-lab2/.env", "../.env", "../../.env"]:
    if os.path.exists(env_candidate):
        load_dotenv(env_candidate)
        break

api_key = os.getenv("GEMINI_API_KEY")

if api_key and api_key != "your_gemini_api_key_here":
    print("=" * 60)
    print(f"[OK] API Key Check: Loaded successfully from .env!")
    print(f"     Key Preview: {api_key[:4]}...{api_key[-4:]} (length: {len(api_key)} chars)")
    print("=" * 60)
else:
    print("[WARNING] No valid GEMINI_API_KEY found. Please check your .env file!")
```

Out [1]:

```
============================================================
[OK] API Key Check: Loaded successfully from .env!
     Key Preview: AQ.A...lDNw (length: 53 chars)
============================================================
```

Step 1.3: Loading the Source PDF Document

Our source of truth for this lab is `streamlit_developer_guide.pdf`, located in our `data/` directory.

We use LangChain's `PyPDFLoader` to read the PDF. It extracts each page as a distinct document object, preserving page numbers so we know exactly where each answer comes from.

In [2]:

```
# Step 1.3: Load the Streamlit Developer Guide into LangChain document objects
from langchain_community.document_loaders import PyPDFLoader
import os
import warnings
warnings.filterwarnings("ignore")

# Search common relative locations so this works from workspace root or notebooks/ folder
pdf_candidates = [
    "../data/streamlit_developer_guide.pdf",
    "data/streamlit_developer_guide.pdf",
    "streamlit-rag-lab2/data/streamlit_developer_guide.pdf"
]
pdf_path = next((p for p in pdf_candidates if os.path.exists(p)), None)
if not pdf_path:
    raise FileNotFoundError("Could not find 'streamlit_developer_guide.pdf'! Check data/ folder.")

loader = PyPDFLoader(pdf_path)
documents = loader.load()

print("=" * 60)
print(f"[DOC LOADER] Successfully loaded {len(documents)} pages from '{pdf_path}'")
print(f"             Page 1 Character Count: {len(documents[0].page_content)} characters")
print("-" * 60)
print("Preview of Page 1 Content:")
print("   " + documents[0].page_content[:220].replace('\n', ' '))
print("=" * 60)
```

Out [2]:

```
============================================================
[DOC LOADER] Successfully loaded 11 pages from '../data/streamlit_developer_guide.pdf'
             Page 1 Character Count: 4042 characters
------------------------------------------------------------
Preview of Page 1 Content:
   STREAMLIT CORE FRAMEWORK & DEVELOPER REFERENCE GUIDE A Technical Manual for Interactive Data
Applications and RAG Evaluation Academic Edition for DS314: Generative AI -- Laboratory Task 2 Release:
2026 Reference Build |
============================================================
```

Step 1.4: Splitting the Guide into Searchable Text Chunks

Why can't we just dump all 11 pages into Gemini at once? In real-world RAG systems, feeding entire books or huge documents is slower, wastes API tokens, and makes it harder for the LLM to focus on the specific paragraph that answers the user's question.

We use `RecursiveCharacterTextSplitter` to break the text down:

- `chunk_size = 1000` **characters**: Each piece is around 150 to 200 words (about one solid paragraph).
- `chunk_overlap = 100` **characters**: A small overlap between adjacent chunks so sentences aren't sliced in half at the boundary.

In [3]:

```python
# Step 1.4: Break down our pages into bite-sized overlapping chunks
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=100
)

docs = text_splitter.split_documents(documents)

print("=" * 60)
print(f"[TEXT SPLITTER] Partitioned {len(documents)} pages into {len(docs)} focused chunks.")
print(f"                Settings: Chunk Size = 1000 characters | Chunk Overlap = 100 characters")
print("-" * 60)
print("Preview of Chunk 0 (First 180 chars):")
print("   " + docs[0].page_content[:180].replace('\n', ' ') + "...")
print("=" * 60)
```

Out [3]:

```
============================================================
[TEXT SPLITTER] Partitioned 11 pages into 43 focused chunks.
                Settings: Chunk Size = 1000 characters | Chunk Overlap = 100 characters
------------------------------------------------------------
Preview of Chunk 0 (First 180 chars):
   STREAMLIT CORE FRAMEWORK & DEVELOPER REFERENCE GUIDE A Technical Manual for Interactive Data
Applications and RAG Evaluation Academic Edition for DS314: Generative AI -- Laboratory...
============================================================
```

Step 1.5: Generating Embeddings & Building the Chroma Vector Store

Computers cannot search text by "meaning" directly—they understand numbers. We use Google's `gemini-embedding-2` to convert every chunk of text into an **embedding** (a list of numbers representing its conceptual meaning).

Then we save all these embeddings into **ChromaDB**, an embedded vector database. When a student asks a question, Chroma compares the question's embedding with all our chunk embeddings to find the most mathematically similar passages in milliseconds.

In [4]:

```python
# Step 1.5: Convert chunks into Gemini embeddings and index them into ChromaDB
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_chroma import Chroma
from dotenv import load_dotenv
import os
import warnings
warnings.filterwarnings("ignore")

# Ensure environment is loaded
for env_candidate in [".env", "streamlit-rag-lab2/.env", "../.env"]:
    if os.path.exists(env_candidate):
        load_dotenv(env_candidate)
        break

embedding_model = GoogleGenerativeAIEmbeddings(
    model="gemini-embedding-2",
    google_api_key=os.getenv("GEMINI_API_KEY")
)

vector_store = Chroma.from_documents(
    docs,
    embedding=embedding_model
)

print("=" * 60)
print(f"[VECTOR STORE] ChromaDB successfully created!")
print(f"               Total Chunks Indexed: {vector_store._collection.count()}")
print(f"               Embedding Model: gemini-embedding-2 (Google GenAI)")
print("=" * 60)
```

Out [4]:

```
============================================================
[VECTOR STORE] ChromaDB successfully created!
               Total Chunks Indexed: 43
               Embedding Model: gemini-embedding-2 (Google GenAI)
============================================================
```

Step 1.6: Initializing the Gemini Chat Model

Next, we initialize Google's **Gemini** (`gemini-3.7-flash`) using LangChain's `ChatGoogleGenerativeAI` .

`gemini-3.7-flash` is optimized for grounded retrieval tasks with fast latency and ample daily quota, keeping the model strictly tied to the retrieved document facts rather than being creative.

In [5]:

```
# Step 1.6: Set up our Gemini language model for grounded factual answers
from langchain_google_genai import ChatGoogleGenerativeAI
from dotenv import load_dotenv
import os
import warnings
import logging

warnings.filterwarnings("ignore")
logging.getLogger("google_genai").setLevel(logging.ERROR)
logging.getLogger("google_genai.models").setLevel(logging.ERROR)

for env_candidate in [".env", "streamlit-rag-lab2/.env", "../.env"]:
    if os.path.exists(env_candidate):
        load_dotenv(env_candidate)
        break

# Use gemini-3.7-flash for responsive, grounded factual generation
llm = ChatGoogleGenerativeAI(
    model="gemini-3.7-flash",
    google_api_key=os.getenv("GEMINI_API_KEY")
)

print("=" * 60)
print("[LLM READY] Google Gemini 3.7 Flash initialized successfully!")
print("            Configured for grounded factual retrieval.")
print("=" * 60)
```

Out [5]:

```
============================================================
[LLM READY] Google Gemini 3.7 Flash initialized successfully!
            Configured for grounded factual retrieval.
============================================================
```

Step 1.7: Assembling the Full RAG Question-Answering Chain

Now we connect all our pieces into an end-to-end runnable pipeline:

1. **The Retriever**: Searches ChromaDB and pulls the **top 2 most relevant chunks** (`k = 2`) for any incoming question.
2. **The Grounded Prompt**: Feeds those chunks into a `<context>` tag and explicitly orders Gemini to answer *only* from the context. If the answer isn't in the document, it must say so.
3. **The Document Chain**: Combines the prompt template with Gemini.
4. **The Retrieval Chain**: Glues the retriever and document chain together so we can simply call `qa_chain.invoke({'input': question})` .

In [6]:

```
# Step 1.7: Wire the retriever, prompt template, and Gemini LLM into our QA chain
from langchain_core.prompts import ChatPromptTemplate
from langchain_classic.chains import create_retrieval_chain
from langchain_classic.chains.combine_documents import create_stuff_documents_chain
import warnings
warnings.filterwarnings("ignore")

# 1. Create retriever returning top-2 chunks
retriever = vector_store.as_retriever(
    search_kwargs={"k": 2}
)

# 2. Define our grounded prompt template with clear boundary rules
prompt = ChatPromptTemplate.from_template(
    """Answer the question based only on the following context.

<context>
{context}
</context>

Question: {input}

If the answer cannot be found in the context, say that the information is not available in the provided
)

# 3. Combine retrieved document chunks with the Gemini model
document_chain = create_stuff_documents_chain(
    llm,
    prompt
)

# 4. Assemble the end-to-end RAG chain
qa_chain = create_retrieval_chain(
    retriever,
    document_chain
)

print("=" * 60)
print("[RAG PIPELINE] End-to-end Q&A chain assembled successfully!")
print("               Retriever: ChromaDB (k=2 chunks)")
print("               Generator: Gemini 3.6 Flash")
print("               Guardrail: Grounded system prompt with explicit boundary instruction")
print("=" * 60)
```

Out [6]:

```
============================================================
[RAG PIPELINE] End-to-end Q&A chain assembled successfully!
               Retriever: ChromaDB (k=2 chunks)
               Generator: Gemini 3.6 Flash
               Guardrail: Grounded system prompt with explicit boundary instruction
============================================================
```

Part 2: Testing the Chatbot (10 Domain Questions)

Now each partner tests the chatbot with 5 specific questions from the Streamlit guide. For each question, we look at:

1. The question asked
2. The answer Gemini generated
3. The actual facts written in the document (Ground Truth)

Partner 1 Test Suite: Display Elements, Widgets & Layouts

Partner 1 -- Question 1

- **Question:** How does a Streamlit app run when a user interacts with an input widget?
- **Document Ground Truth (Section 1.2):** Streamlit reruns the entire Python script from top to bottom every time a user interacts with an input widget.

In [7]:

```
# Partner 1 - Question 1
q_p1_1 = "How does a Streamlit app run when a user interacts with an input widget?"
expected_p1_1 = "Section 1.2: Streamlit reruns the entire Python script from top to bottom every time a
result_p1_1 = qa_chain.invoke({"input": q_p1_1})

print("=" * 60)
print("[QUESTION - Partner 1 - Q1]: How does a Streamlit app run when a user interacts with an input wi
print("-" * 60)
print(f"[MODEL ANSWER]:\n{result_p1_1['answer']}")
print("-" * 60)
print(f"[DOCUMENT GROUND TRUTH]:\n{expected_p1_1}")
print("=" * 60)
```

Out [7]:

```
============================================================
[QUESTION - Partner 1 - Q1]: How does a Streamlit app run when a user interacts with an input widget?
------------------------------------------------------------
[MODEL ANSWER]:
Based on the provided context, when a user interacts with an input widget, Streamlit executes the entire
Python script from top to bottom (triggering an automatic re-evaluation of the entire script).
------------------------------------------------------------
[DOCUMENT GROUND TRUTH]:
Section 1.2: Streamlit reruns the entire Python script from top to bottom every time a user interacts
with an input widget.
============================================================
```

Partner 1 -- Question 2

- **Question:** Which Streamlit function is called the 'Swiss Army knife' for showing text, numbers, DataFrames, and charts?
- **Document Ground Truth (Section 2.3):** `st.write()` is Streamlit's Swiss Army knife because it can display many different Python data types automatically.

In [8]:

```
# Partner 1 - Question 2
q_p1_2 = "Which Streamlit function is called the 'Swiss Army knife' for showing text, numbers, DataFram
expected_p1_2 = "Section 2.3: st.write() is Streamlit's Swiss Army knife because it can display many di
result_p1_2 = qa_chain.invoke({"input": q_p1_2})

print("=" * 60)
print("[QUESTION - Partner 1 - Q2]: Which Streamlit function is called the 'Swiss Army knife' for showi
print("-" * 60)
print(f"[MODEL ANSWER]:\n{result_p1_2['answer']}")
print("-" * 60)
print(f"[DOCUMENT GROUND TRUTH]:\n{expected_p1_2}")
print("=" * 60)
```

Out [8]:

```
============================================================
[QUESTION - Partner 1 - Q2]: Which Streamlit function is called the 'Swiss Army knife' for showing text,
numbers, DataFrames, and charts?
------------------------------------------------------------
[MODEL ANSWER]:
Based on the provided context, the **`st.write()`** function is celebrated as Streamlit's 'Swiss Army
knife'.
------------------------------------------------------------
[DOCUMENT GROUND TRUTH]:
Section 2.3: st.write() is Streamlit's Swiss Army knife because it can display many different Python
data types automatically.
============================================================
```

Partner 1 -- Question 3

- **Question:** What does `st.button()` return, and how long does that return value stay True?
- **Document Ground Truth (Section 2.5):** `st.button()` returns a boolean `True` only during the single script rerun right after a click. On the next rerun, it resets back to `False`.

In [9]:

```
# Partner 1 - Question 3
q_p1_3 = "What does st.button() return, and how long does that return value stay True?"
expected_p1_3 = "Section 2.5: st.button() returns a boolean True only during the single script rerun ri
result_p1_3 = qa_chain.invoke({"input": q_p1_3})

print("=" * 60)
print("[QUESTION - Partner 1 - Q3]: What does st.button() return, and how long does that return value s
print("-" * 60)
print(f"[MODEL ANSWER]:\n{result_p1_3['answer']}")
print("-" * 60)
print(f"[DOCUMENT GROUND TRUTH]:\n{expected_p1_3}")
print("=" * 60)
```

Out [9]:

```
============================================================
[QUESTION - Partner 1 - Q3]: What does st.button() return, and how long does that return value stay
True?
------------------------------------------------------------
[MODEL ANSWER]:
Based on the provided context:

* **What it returns:** `st.button()` returns a boolean value.
* **How long it stays True:** It evaluates to `True` only during the exact script rerun immediately
following the user's click event, and it automatically reverts back to `False` on the subsequent rerun.
------------------------------------------------------------
[DOCUMENT GROUND TRUTH]:
Section 2.5: st.button() returns a boolean True only during the single script rerun right after a click.
On the next rerun, it resets back to False.
============================================================
```

Partner 1 -- Question 4

- **Question:** How can a developer hide or mask sensitive input like passwords in `st.text_input`?
- **Document Ground Truth (Section 2.8):** Set `type='password'` inside `st.text_input` (for example: `st.text_input('Password', type='password')`).

In [10]:

```
# Partner 1 - Question 4
q_p1_4 = "How can a developer hide or mask sensitive input like passwords in st.text_input?"
expected_p1_4 = "Section 2.8: Set type='password' inside st.text_input (for example: st.text_input('Pas
result_p1_4 = qa_chain.invoke({"input": q_p1_4})

print("=" * 60)
print("[QUESTION - Partner 1 - Q4]: How can a developer hide or mask sensitive input like passwords in
print("-" * 60)
print(f"[MODEL ANSWER]:\n{result_p1_4['answer']}")
print("-" * 60)
print(f"[DOCUMENT GROUND TRUTH]:\n{expected_p1_4}")
print("=" * 60)
```

Out [10]:

```
============================================================
[QUESTION - Partner 1 - Q4]: How can a developer hide or mask sensitive input like passwords in
st.text_input?
------------------------------------------------------------
[MODEL ANSWER]:
Based on the provided context, a developer can mask sensitive input by explicitly configuring the `type`
parameter and setting `type='password'`. This causes the browser to render obscuring bullet dots instead
of cleartext characters.
------------------------------------------------------------
[DOCUMENT GROUND TRUTH]:
Section 2.8: Set type='password' inside st.text_input (for example: st.text_input('Password',
type='password')).
============================================================
```

Partner 1 -- Question 5

- **Question:** Which function creates side-by-side columns in Streamlit, and what parameter sets their widths?
- **Document Ground Truth (Section 2.9):** `st.columns()` creates columns, and the `spec` parameter controls the number of columns and their proportional widths.

In [11]:

```
# Partner 1 - Question 5
q_p1_5 = "Which function creates side-by-side columns in Streamlit, and what parameter sets their width
expected_p1_5 = "Section 2.9: st.columns() creates columns, and the spec parameter controls the number
result_p1_5 = qa_chain.invoke({"input": q_p1_5})

print("=" * 60)
print("[QUESTION - Partner 1 - Q5]: Which function creates side-by-side columns in Streamlit, and what
print("-" * 60)
print(f"[MODEL ANSWER]:\n{result_p1_5['answer']}")
print("-" * 60)
print(f"[DOCUMENT GROUND TRUTH]:\n{expected_p1_5}")
print("=" * 60)
```

Out [11]:

```
============================================================
[QUESTION - Partner 1 - Q5]: Which function creates side-by-side columns in Streamlit, and what
parameter sets their widths?
------------------------------------------------------------
[MODEL ANSWER]:
Based on the provided context, the function that creates side-by-side multi-column layouts is
**`st.columns`** (or `st.columns(spec, gap='small')`), and the parameter that sets their relative
proportional widths is **`spec`**.
------------------------------------------------------------
[DOCUMENT GROUND TRUTH]:
Section 2.9: st.columns() creates columns, and the spec parameter controls the number of columns and
their proportional widths.
============================================================
```

Partner 2 Test Suite: State Management, Caching & Configuration

Partner 2 -- Question 6

- **Question:** Why do we need `st.session_state` instead of regular Python variables to save data between user clicks?
- **Document Ground Truth (Section 3.1):** Because Streamlit reruns the whole script from the beginning on every click, which resets normal variables. `st.session_state` preserves values across these reruns.

In [12]:

```
# Partner 2 - Question 6
q_p2_6 = "Why do we need st.session_state instead of regular Python variables to save data between user
expected_p2_6 = "Section 3.1: Because Streamlit reruns the whole script from the beginning on every cli
result_p2_6 = qa_chain.invoke({"input": q_p2_6})

print("=" * 60)
print("[QUESTION - Partner 2 - Q6]: Why do we need st.session_state instead of regular Python variables
print("-" * 60)
print(f"[MODEL ANSWER]:\n{result_p2_6['answer']}")
print("-" * 60)
print(f"[DOCUMENT GROUND TRUTH]:\n{expected_p2_6}")
print("=" * 60)
```

Out [12]:

```
============================================================
[QUESTION - Partner 2 - Q6]: Why do we need st.session_state instead of regular Python variables to save
data between user clicks?
------------------------------------------------------------
[MODEL ANSWER]:
Based on the provided context, `st.session_state` is needed because Streamlit's default execution model
reruns the entire Python script from line 1 on every widget interaction. Under standard Python scoping
rules, all local variables created during a run are destroyed when script execution terminates.

Consequently, if standard Python variables are used to accumulate or save data, they will be reset to
their default initialization values on every single click. `st.session_state` solves this by acting as a
dedicated, stateful, session-scoped dictionary-like object that persists data across reruns.
------------------------------------------------------------
[DOCUMENT GROUND TRUTH]:
Section 3.1: Because Streamlit reruns the whole script from the beginning on every click, which resets
normal variables. st.session_state preserves values across these reruns.
============================================================
```

Partner 2 -- Question 7

- **Question:** What is the main difference between `@st.cache_data` and `@st.cache_resource` when returning cached objects?
- **Document Ground Truth (Sections 3.6 & 3.8):** `@st.cache_data` returns a fresh copy of the cached data on every call so you don't mutate it. `@st.cache_resource` returns the exact same shared object (singleton) without making copies.

In [13]:

```
# Partner 2 - Question 7
q_p2_7 = "What is the main difference between @st.cache_data and @st.cache_resource when returning cach
expected_p2_7 = "Section 3.6 & 3.8: @st.cache_data returns a fresh copy of the cached data on every cal
result_p2_7 = qa_chain.invoke({"input": q_p2_7})

print("=" * 60)
print("[QUESTION - Partner 2 - Q7]: What is the main difference between @st.cache_data and @st.cache_re
print("-" * 60)
print(f"[MODEL ANSWER]:\n{result_p2_7['answer']}")
print("-" * 60)
print(f"[DOCUMENT GROUND TRUTH]:\n{expected_p2_7}")
print("=" * 60)
```

Out [13]:

```
============================================================
[QUESTION - Partner 2 - Q7]: What is the main difference between @st.cache_data and @st.cache_resource
when returning cached objects?
------------------------------------------------------------
[MODEL ANSWER]:
Based on the provided context, the main difference when returning cached objects is their data handling
strategy:

* **`@st.cache_data`** serializes return values and returns an isolated, new serializable copy on each
call.
* **`@st.cache_resource`** returns the identical global singleton reference without copying.
------------------------------------------------------------
[DOCUMENT GROUND TRUTH]:
Section 3.6 & 3.8: @st.cache_data returns a fresh copy of the cached data on every call so you don't
mutate it. @st.cache_resource returns the exact same shared object (singleton) without making copies.
============================================================
```

Partner 2 -- Question 8

- **Question:** Which cache decorator should you use for heavy objects like machine learning models or database connections?
- **Document Ground Truth (Sections 3.7 & 3.8):** Use `@st.cache_resource` for heavy global objects like ML models, database connection pools, or network clients.

In [14]:

```
# Partner 2 - Question 8
q_p2_8 = "Which cache decorator should you use for heavy objects like machine learning models or databa
expected_p2_8 = "Section 3.7 & 3.8: Use @st.cache_resource for heavy global objects like ML models, dat
result_p2_8 = qa_chain.invoke({"input": q_p2_8})

print("=" * 60)
print("[QUESTION - Partner 2 - Q8]: Which cache decorator should you use for heavy objects like machine
print("-" * 60)
print(f"[MODEL ANSWER]:\n{result_p2_8['answer']}")
print("-" * 60)
print(f"[DOCUMENT GROUND TRUTH]:\n{expected_p2_8}")
print("=" * 60)
```

Out [14]:

```
============================================================
[QUESTION - Partner 2 - Q8]: Which cache decorator should you use for heavy objects like machine
learning models or database connections?
------------------------------------------------------------
[MODEL ANSWER]:
Based on the provided context, you should use the **`@st.cache_resource`** decorator for machine
learning model instances/weights and database connection pools/clients.
------------------------------------------------------------
[DOCUMENT GROUND TRUTH]:
Section 3.7 & 3.8: Use @st.cache_resource for heavy global objects like ML models, database connection
pools, or network clients.
============================================================
```

Partner 2 -- Question 9

- **Question:** Where must `st.set_page_config()` be placed in your script, and what happens if you put it after a display element?
- **Document Ground Truth (Section 3.9):** It must be the very first Streamlit command called in the script. If you call it after visual elements, Streamlit throws an error (`StreamlitAPIException`).

In [15]:

```
# Partner 2 - Question 9
q_p2_9 = "Where must st.set_page_config() be placed in your script, and what happens if you put it afte
expected_p2_9 = "Section 3.9: It must be the very first Streamlit command called in the script. If you
result_p2_9 = qa_chain.invoke({"input": q_p2_9})

print("=" * 60)
print("[QUESTION - Partner 2 - Q9]: Where must st.set_page_config() be placed in your script, and what
print("-" * 60)
print(f"[MODEL ANSWER]:\n{result_p2_9['answer']}")
print("-" * 60)
print(f"[DOCUMENT GROUND TRUTH]:\n{expected_p2_9}")
print("=" * 60)
```

Out [15]:

```
============================================================
[QUESTION - Partner 2 - Q9]: Where must st.set_page_config() be placed in your script, and what happens
if you put it after a display element?
------------------------------------------------------------
[MODEL ANSWER]:
Based on the provided context:

* **Where it must be placed:** `st.set_page_config()` must be the very first Streamlit command executed
in your Python script, preceding any other calls to `st.write()`, `st.title()`, `st.sidebar`, or widget
definitions.
* **What happens if placed after a visual element:** Streamlit will raise an immediate
`StreamlitAPIException` and halt the execution of the script.
------------------------------------------------------------
[DOCUMENT GROUND TRUTH]:
Section 3.9: It must be the very first Streamlit command called in the script. If you call it after
visual elements, Streamlit throws an error (StreamlitAPIException).
============================================================
```

Partner 2 -- Question 10

- **Question:** What layout choices are available for the `layout` parameter in `st.set_page_config()`?
- **Document Ground Truth (Section 3.9):** The allowed options are exactly `'centered'` (standard centered width) and `'wide'` (full screen width).

In [16]:

```
# Partner 2 - Question 10
q_p2_10 = "What layout choices are available for the layout parameter in st.set_page_config()?"
expected_p2_10 = "Section 3.9: The allowed options are exactly 'centered' (standard centered width) and
result_p2_10 = qa_chain.invoke({"input": q_p2_10})

print("=" * 60)
print("[QUESTION - Partner 2 - Q10]: What layout choices are available for the layout parameter in st.s
print("-" * 60)
print(f"[MODEL ANSWER]:\n{result_p2_10['answer']}")
print("-" * 60)
print(f"[DOCUMENT GROUND TRUTH]:\n{expected_p2_10}")
print("=" * 60)
```

Out [16]:

```
============================================================
[QUESTION - Partner 2 - Q10]: What layout choices are available for the layout parameter in
st.set_page_config()?
------------------------------------------------------------
[MODEL ANSWER]:
Based on the provided context, the allowed choices for the `layout` parameter are:

* **`'centered'`**: The default standard width constraint.
* **`'wide'`**: Utilizes the entire horizontal width of the screen.
------------------------------------------------------------
[DOCUMENT GROUND TRUTH]:
Section 3.9: The allowed options are exactly 'centered' (standard centered width) and 'wide' (full
screen width).
============================================================
```

Part 3: The Trap Questions (Testing System Limits)

3.1 What is a Trap Question?

In RAG, a **trap question** asks about something plausible that does **not** exist in the source document. We use it to test if our chatbot stays grounded or if it hallucinates (makes up fake answers).

We test **four deliberate trap questions**, mapping directly to the explicit architectural omissions documented in **Section 4.2 of the Streamlit Developer Guide**:

1. **Trap 1 (CLI Cloud Deployment):** "*What is the command syntax for using st.deploy() to automatically containerize and deploy a Streamlit app to AWS or Google Cloud?*"
   - **What the Document Says (Sections 4.2 & 4.3):** Streamlit core has **no** `st.deploy()` command or built-in cloud deployment syntax. Deploying to the cloud requires external tools like Docker.
2. **Trap 2 (Distributed State Persistence):** "*How do you configure st.session_state to synchronize and persist user session variables across multiple server nodes using a distributed Redis or Memcached cluster?*"
   - **What the Document Says (Section 4.2):** Streamlit core strictly holds session state in local server process memory. It does **not** support distributed Redis or Memcached synchronization across multi-node load-balanced clusters.
3. **Trap 3 (Native User Authentication Database):** "*What is the parameter syntax for st.login() to authenticate users against a built-in PostgreSQL database or OAuth2 provider?*"
   - **What the Document Says (Section 4.2):** Streamlit core lacks native OAuth2 databases, user registration workflows, or JWT token verification servers. There is no built-in `st.login()` function in the core library.
4. **Trap 4 (Automated Containerization Engine):** "*Which native Streamlit command automatically generates a Dockerfile and Kubernetes pod manifest for containerizing the application?*"
   - **What the Document Says (Sections 4.2 & 4.4):** Streamlit core does **not** generate Dockerfiles, OCI manifests, or Kubernetes pod specs natively. Containerization must be authored manually with external tooling.

- **Expected Behavior:** The chatbot should honestly state that this information is not in the document, rather than inventing fake commands, methods, or syntax.

In [17]:

```
# Step 3.1: Test our 4 trap questions on the standard RAG chain
trap_tests = [
    {
        "id": "Trap 1 (CLI Cloud Deployment)",
        "question": "What is the command syntax for using st.deploy() to automatically containerize and
        "expected": "Section 4.2 & 4.3: st.deploy() does not exist in Streamlit core. Deployments requi
    },
    {
        "id": "Trap 2 (Distributed State Persistence)",
        "question": "How do you configure st.session_state to synchronize and persist user session vari
        "expected": "Section 4.2: Streamlit core strictly holds state in local process memory; no distr
    },
    {
        "id": "Trap 3 (Native User Authentication Database)",
        "question": "What is the parameter syntax for st.login() to authenticate users against a built-
        "expected": "Section 4.2: Streamlit core has no native st.login() command, OAuth2 database, use
    },
    {
        "id": "Trap 4 (Automated Containerization Engine)",
        "question": "Which native Streamlit command automatically generates a Dockerfile and Kubernetes
        "expected": "Section 4.2 & 4.4: Streamlit core does not generate Dockerfiles or Kubernetes pod
    }
]

print("=" * 60)
print("RUNNING BATCH TRAP QUESTION GROUNDING EVALUATION")
print("=" * 60)

for test in trap_tests:
    result = qa_chain.invoke({"input": test["question"]})
    print(f"\n[{test['id'].upper()}]")
    print(f"Question: {test['question']}")
    print("-" * 60)
    print(f"[MODEL ANSWER]:\n{result['answer']}")
    print("-" * 60)
    print(f"[DOCUMENT GROUND TRUTH]:\n{test['expected']}")
    print("=" * 60)
```

Out [17]:

```
============================================================
RUNNING BATCH TRAP QUESTION GROUNDING EVALUATION
============================================================

[TRAP 1 (CLI CLOUD DEPLOYMENT)]
Question: What is the command syntax for using st.deploy() to automatically containerize and deploy a
Streamlit app to AWS or Google Cloud?
------------------------------------------------------------
[MODEL ANSWER]:
Based on the provided document, there is no command syntax for `st.deploy()`. The context states that
any inquiry asking how to configure an internal `st.deploy()` method represents an unanswerable inquiry
outside the boundaries of the framework, as packaging and hosting a Streamlit application requires
external DevOps tooling (such as custom Dockerfiles and cloud container platforms).

Therefore, the requested information is not available in the provided document.
------------------------------------------------------------
[DOCUMENT GROUND TRUTH]:
Section 4.2 & 4.3: st.deploy() does not exist in Streamlit core. Deployments require external tools like
Docker.
============================================================

[TRAP 2 (DISTRIBUTED STATE PERSISTENCE)]
Question: How do you configure st.session_state to synchronize and persist user session variables across
multiple server nodes using a distributed Redis or Memcached cluster?
------------------------------------------------------------
[MODEL ANSWER]:
```

```
The information is not available in the provided document.
------------------------------------------------------------
[DOCUMENT GROUND TRUTH]:
Section 4.2: Streamlit core strictly holds state in local process memory; no distributed Redis or
Memcached clustering exists.
============================================================

[TRAP 3 (NATIVE USER AUTHENTICATION DATABASE)]
Question: What is the parameter syntax for st.login() to authenticate users against a built-in
PostgreSQL database or OAuth2 provider?
------------------------------------------------------------
[MODEL ANSWER]:
The information is not available in the provided document.
------------------------------------------------------------
[DOCUMENT GROUND TRUTH]:
Section 4.2: Streamlit core has no native st.login() command, OAuth2 database, user registration system,
or JWT verification server.
============================================================

[TRAP 4 (AUTOMATED CONTAINERIZATION ENGINE)]
Question: Which native Streamlit command automatically generates a Dockerfile and Kubernetes pod
manifest for containerizing the application?
------------------------------------------------------------
[MODEL ANSWER]:
Based on the provided context, the information is not available in the provided document. The text
indicates that packaging and hosting a Streamlit application requires external DevOps tooling (such as
creating a custom Dockerfile) and does not provide a native Streamlit command for automatically
generating Dockerfiles or Kubernetes manifests.
------------------------------------------------------------
[DOCUMENT GROUND TRUTH]:
Section 4.2 & 4.4: Streamlit core does not generate Dockerfiles or Kubernetes pod specs natively;
packaging requires external DevOps tools.
============================================================
```

3.2 Trap Questions Assessment Matrix

Trap Test	Target Fallacy	Did Model Hallucinate?	Document Ground Truth Boundary	Observation
Trap 1: Cloud Deployment	`st.deploy()` CLI syntax	**No (Grounded Refusal)**	Section 4.2 & 4.3: Streamlit has no `st.deploy()` or cloud deployment CLI commands.	Correctly refused; stated info is not available.
Trap 2: Distributed State	Redis / Memcached clustering	**No (Grounded Refusal)**	Section 4.2: Session state is strictly local in-process memory; no Redis sync.	Correctly refused; avoided inventing fake sync methods.
Trap 3: User Auth Database	Built-in `st.login()` / OAuth DB	**No (Grounded Refusal)**	Section 4.2: Lacks native OAuth2 databases, auth tables, or JWT verification.	Correctly refused; did not hallucinate login parameters.
Trap 4: Auto-Containerization	Auto Dockerfile / K8s manifests	**No (Grounded Refusal)**	Section 4.2 & 4.4: Does not generate Dockerfiles or Kubernetes pod specs natively.	Correctly refused; acknowledged DevOps tooling is external.

Part 4: Individual Student Reflections

Partner 1 Reflection (150 -- 250 words)

For this lab activity, Rogelio and I teamed up to explore two different projects, with this notebook focusing on the Streamlit guide. While we tested each other's ideas across our work, my job in this notebook was to test the first five questions about basic Streamlit tools and how apps run.

The chatbot was really good at finding facts from the PDF. For example, it correctly answered that Streamlit reruns the whole script from top to bottom whenever a user clicks something. It also found that st.write() is called the Swiss Army knife, and it showed the exact way to hide passwords using type='password' in st.text_input(). It even got tricky details right, like how st.button() only stays True for a single rerun.

Testing also showed the limits of the system. If my question was too broad, the search had to pull extra pieces of text to find the answer. We also tested trap questions about things like st.deploy(), which do not exist in Streamlit. The bot did not make up fake code. Instead, it told the truth and said the information was not in the guide. Overall, this lab showed me that RAG keeps the AI honest and stops it from guessing.

Partner 2 Reflection (150 -- 250 words)

Working on this lab activity alongside my buddy AJ was a great learning experience. We chose two different topics - I built my RAG chatbot on the Streamlit Developer Guide, while AJ built his on Transfory (a PyPI package I made last year). This setup allowed us to test each other's chatbots, compare retrieval across two different technical documents, and share helpful feedback.

I want to be transparent that I used Generative AI to help write the code and fix errors. However, I didn't use it as a shortcut. I treated the AI like a tutor so I could experiment, test code cells, and understand what was happening under the hood.

My assigned test questions (Questions 6 to 10) focused on state management, caching, and configuration. The results showed that our ChromaDB retriever did a great job pulling accurate facts. The chatbot correctly explained why we need `st.session_state` to persist variables across reruns, and it clearly separated `@st.cache_data` (which returns fresh copies) from `@st.cache_resource` (which shares single objects for heavy ML models). It even accurately caught that putting `st.set_page_config()` after visual elements throws a StreamlitAPIException.

Testing the trap question on `st.deploy()` also showed the limits of the system. Instead of making up fake deployment commands, the model stayed grounded and admitted the command was not in the guide. Overall, testing my questions helped me genuinely understand both RAG systems and Streamlit development.